

Algorithms and Analysis Report

COSC2123/3119

Graph Algorithms in Action: Modelling a Disease Outbreak

Shenghang Weng

S4165092

Assessment Type	Individual assignment. Submit online via GitHub.
Due Date	Week 11, Thursday May 21, 13:00 (1pm). A late penalty will apply to assessments submitted after 13:00.
Marks	30

1 Learning Outcomes

This assessment relates to four learning outcomes of the course which are:

- CLO 1: Compare, contrast, and apply the key algorithmic design paradigms: brute force, divide and conquer, decrease and conquer, transform and conquer, greedy, dynamic programming and iterative improvement;
- CLO 2: Compare, contrast, and apply key data structures: trees, lists, stacks, queues, hash tables and graph representations;
- CLO 3: Define, compare, analyse, and solve general algorithmic problem types: sorting, searching, graphs and geometric;
- CLO 4: Theoretically compare and analyse the time complexities of algorithms and data structures; and
- CLO 5: Implement, empirically compare, and apply fundamental algorithms and data structures to real-world problems.

2 Overview

Across multiple tasks in this assignment, you will model a disease outbreak in the city of Metropolis as a weighted graph to capture residents' contacts, implement and compare algorithms for estimating infection risk through the population, and design a dynamic programming strategy to allocate scarce antiviral treatment as effectively as possible. Some components ask you to critically assess your solutions through both theoretical analysis and

controlled empirical experiments, encouraging reflection on the relationship between algorithm design and real-world performance.

*I certify that this is all my own original work. If I took any parts from elsewhere, then they were non-essential parts of the assignment, and they are clearly attributed in my submission. I will show I agree to this honor code by typing "Yes": **YOUR ANSWER HERE: YES***

3 Motivation

The T-Virus has broken out in the city of Metropolis. As the newly appointed Regional Health Director, you have been handed a dossier containing everything the epidemiologists know: a map of every known contact between residents, the probability that each contact transmits the virus in a single day, and the vulnerability of each resident. The clock is ticking — every day the virus spreads further, and your antiviral stockpile is limited.

Your job has four parts:

1. Build the contact network (Task A) — represent the city's contact relationships as a graph, efficiently enough to run algorithms on in real time (implement `graph/adjacency_matrix.py`).
2. Estimate infection risk (Task B) — compute the probability that each resident becomes infected over the coming T days, identifying the most at-risk residents before the outbreak grows (implement `transmission/task_b.py` and write report).
3. Analyse your solution (Task C) — theoretically and empirically compare your algorithm against the provided baseline across different network structures (write report on analysis of task A and B).
4. Allocate antiviral treatment (Task D) — decide who receives the scarce supply of doses, maximising the benefit to the population within the limits of your stockpile (implement `treatment/task_d.py` and write report).

Figure 1 shows the contact network for a sub-population in Metropolis. Each resident is a node, and each contact relationship is an edge labelled with the daily probability of transmission. Patient zero V_0 is already infected — the question is how far the virus will spread, and who we should protect first.

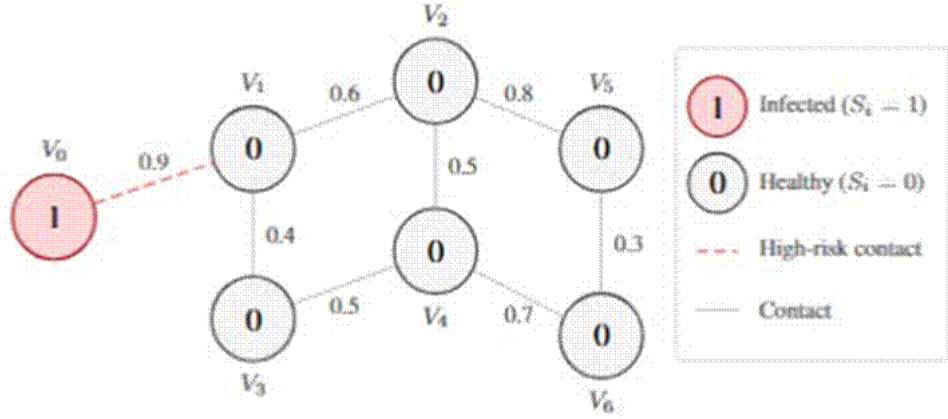


Figure 1: The contact network $G = (V, E)$ for Metropolis. V_0 is patient zero. Edge labels give the daily transmission probability $w_{ij} \in (0, 1]$. Dashed red edges originate from an infected resident.

By vaccinating even a small number of strategically chosen residents, we can sever the most dangerous transmission pathways through the network. Figure 2 shows an example: vaccinating V_1 — the single resident through whom all transmission from V_0 must pass — protects every downstream resident at once.

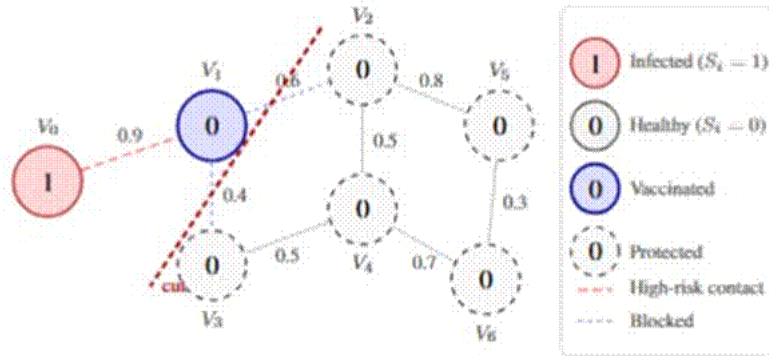


Figure 2: Vaccinating V_1 — the single bottleneck through whom all transmission from V_0 must pass — severs every onward pathway at once, protecting V_2 through V_6 entirely. This illustrates the core challenge of Task D.

3.1 Mathematical Background

We model the city of Metropolis as a weighted undirected graph $G = (V, E)$, where:

- each vertex $V_i \in V$ represents a resident, with infection state $S_i \in \{0, 1\}$ where $S_i = 0$ is healthy and $S_i = 1$ is infected;
- each edge $(V_i, V_j) \in E$ represents a contact relationship between two residents; and

- each edge carries a weight $w_{ij} \in (0, 1]$, the probability that an infected resident V_i transmits the virus to a healthy neighbour V_j in a single day.

Each resident V_i also has two personal attributes relevant to the vaccine allocation in Task D:

- a dosage requirement $c_i \in \mathbb{Z}^+$, the number of antiviral doses required to vaccinate resident V_i ; and
- a vulnerability score $\phi_i \in (0, 1]$, reflecting how severely resident V_i would be harmed if infected.

Once infected, a resident remains infectious forever. Over a planning horizon of T days, the probability that an infected resident V_i infects neighbour V_j at least once is:

$$p_{ij} = 1 - (1 - w_{ij})^T$$

As $T \rightarrow \infty, p_{ij} \rightarrow 1$ for any $w_{ij} > 0$ — given enough time, every resident reachable from patient zero will eventually be infected. This makes early intervention critical.

Task B: Estimating Infection Risk Over Time (7 marks, 1 page)

Pseudo-code

```

Algorithm Dynamic Programing( $G, s, T$ )
Require: Graph  $G = (V, E)$ ; source  $s$ ; horizon  $T$ ;
Ensure: Estimated risk table where  $table[t][i] \approx r_i$ 
1: initialise  $table[0..T][0..|V|-1] \leftarrow 0.0$ 
2:  $table[0][s] \leftarrow 1.0$ 
3: for  $t = 1$  to  $T$  do
4:   for each vertex  $V_i$  in  $V$  do
5:      $escape \leftarrow 1.0$ 
6:     for each neighbour  $V_j$  of  $V_i$  with edge weight  $w_{ij}$  do
7:        $escape \leftarrow escape \times (1 - table[t-1][j] \times w_{ij})$ 
8:      $table[t][i] \leftarrow 1 - (1 - table[t-1][i]) \times escape$ 
9:    $table[t][s] \leftarrow 1.0$ 
10: return table

```

Limitations of the Recurrence

(a)

The structural property is the presence of cycles in the graph. The recurrence assumes every neighbour of V_i spreads the virus independently. This breaks down when cycles exist, because two neighbours can share a common infection source, making their infection statuses correlated, and not independent, which violates the product rule. This is why the recurrence outputs an approximation rather than an exact answer.

(b)

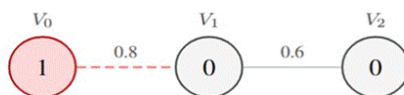


Figure 3: Example of a simple infectivity network chain (Figure 5 from the assignment spec). V_0 is patient zero.

In Figure 3, V_1 at $t = 3$. V_1 has two neighbours: V_0 ($w = 0.8$) and V_2 ($w = 0.6$). The recurrence:

$$r_{1,3} = 1 - (1 - r_{1,2}) \times (1 - r_{0,2} \times 0.8) \times (1 - r_{2,2} \times 0.6)$$

The problem is $(1 - r_{2,2} \times 0.6)$. V_2 can only get infected from V_1 in the first place. So when the formula adds V_2 as a source of risk to V_1 , it believes V_1 might infect V_2 , and then V_2 might infect V_1 back. Leading to an infection looping back to its own source, and the formula treats it as an extra, independent risk, as if V_2 caught the disease from somewhere else entirely. Because the recurrence treats V_2 as an independent source of infection for V_1 , it counts this looped pathway as additional risk that does not truly exist. The true probability would require tracking the joint infection state of all neighbours simultaneously, which the product formula cannot do. As a result, the recurrence overestimates the true infection risk for V_1 at $t=3$.

(c)

The recurrence gives an exact answer for trees, because on a tree, there is only one path between any two vertices, so it prevents 2 neighbours of any vertex V_i to share the same infection source. Thus the infection probability is truly independent of one another. No cycle exists, hence the recurrence gives an exact answer.

Task C: Analysis of Infection Risk Algorithms (8 marks, 2 pages)

1. Algorithm Complexity Analysis

Notation: $|V|$ = residents, $|E|$ = edges, T = planning horizon, X = simulations, $\deg(V_i)$ = degree of resident V_i .

(a) MC + Adj. Matrix — $O(T^2 \cdot X \cdot |V|^2)$ In Algorithm 1, we have the following loops: outer day t , inner simulation x , inner-inner day d (1 to t), then for each resident, scan all neighbours. To find the infected neighbours of a resident using an adjacency matrix, the whole row has to be scanned, which takes $O(|V|)$ for each resident, and $O(|V|^2)$ for the entire population for each (t, x) pair, including pairs that are not an edge. The T^2 factor is from the inner day d going up to t , summing to $T(T+1)/2 \approx T^2$ over all outer days. This leads to $O(T^2 \cdot X \cdot |V|^2)$ in total which I believe will be the slowest out of all.

(b) MC + Adj. List — $O(T^2 \cdot X \cdot |E|)$ The loop structure is the same as (a). The only difference is how neighbours are found. With an adjacency list, each resident V_i visits only its actual neighbours, $\deg(V_i)$ entries instead of $|V|$. Now summing over all residents takes $\sum \deg(V_i) = 2|E|$, so $O(|E|)$ per (t, x) pair, instead of $O(|V|^2)$ for a sparse city network, where $|E| \ll |V|^2$, this is significantly faster. When the density $p \rightarrow 1.0$, $|E|$ approaches $|V|^2$ and the gap closes making the two representations equally expensive.

(c) DP + Adj. Matrix — $O(T \cdot |V|^2)$ The DP fills one column of a $|V| \times T$ table per time step. The escape-probability product must be calculated for each resident at each time step, which involves a full scan of the matrix row, $O(|V|)$ per resident, and $O(|V|^2)$ per time step. There is no X factor since DP is deterministic and runs once. The dominant cost is the cost of paying for the zero entries in sparse graphs.

(d) DP + Adj. List — $O(T \cdot |E|)$ Same DP table as (c), but the product of escape-probability only iterates over real neighbours, which is $O(\deg(V_i))$ per resident, summing to $O(|E|)$ per time step. No X factor, no wasted zero-entry scans. This is the fastest of the four combinations for sparse graphs, and gives exact results.

2. Empirical Design

I used a one-factor-at-a-time approach, which I vary one parameter across a range, while the rest fixed at baseline values ($T=10$, $X=100$, $p=0.1$, $|V|=200$). For each setting, the algorithm was timed and repeated over 5 different random seeds to reduce noise from graph structure variation. Times were averaged across seeds.

These 4 variables and its corresponding inputs is what I've varied and used to test the algorithms:

- $|V| \in \{50, 100, 200, 400, 700\}$ — isolates the $|V|^2$ term in matrix variants. For list variants, $|E| \approx p \cdot |V|^2$, so increasing $|V|$ also grows $|E|$, showing the representation gap.
- $p \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ — matrix runtimes should be flat (always scan full row regardless of density), while list runtimes should grow with $|E|$ and converge to the matrix at $p=1.0$.
- $X \in \{10, 50, 100, 200, 500\}$ — isolates the X factor in MC variants. DP is unaffected by X and should appear as flat horizontal lines.
- $T \in \{2, 5, 10, 20, 40, 80\}$ — on a log-log plot, MC's slope should be ≈ 2 ($O(T^2)$) while DP's slope ≈ 1 ($O(T)$), directly confirming the theoretical gap.

3. Empirical Analysis

All error bars show ± 1 SD across the 5 graph seeds.

Figures 1 & 2 use a log y-axis; Figure 4 is log-log. Baseline values are held fixed when varying any single parameter.

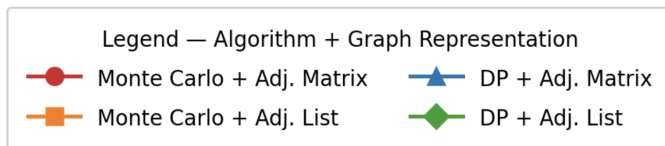


Fig. 1 (Runtime vs. $|V|$): Both matrix variants grow steeply with $|V|$, consistent with the $|V|^2$ term, but MC+Matrix grows faster due to the additional $T \cdot X$ factor. At $|V|=700$, DP+List is approximately 1000× faster than MC+Matrix (DP + list = 0.001, and MC + matrix = 10).

Fig. 2 (Runtime vs. edge density p): Matrix runtimes are flat for all values of p , the full $|V|$ row is always scanned, no matter how many edges there are. List runtimes grow linearly with $|E|$ and converge to the matrix curves at $p=1.0$. The list variants are roughly 3–5× faster at $p=0.05$.

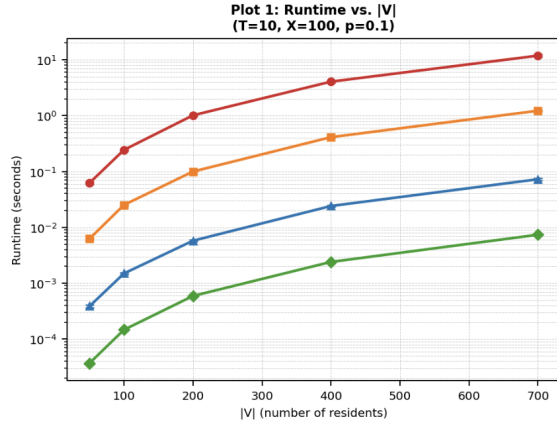


Fig. 1: Runtime vs. $|V|$ (log y-axis)

Fig. 1: Runtime vs. $|V|$ (log y-axis)

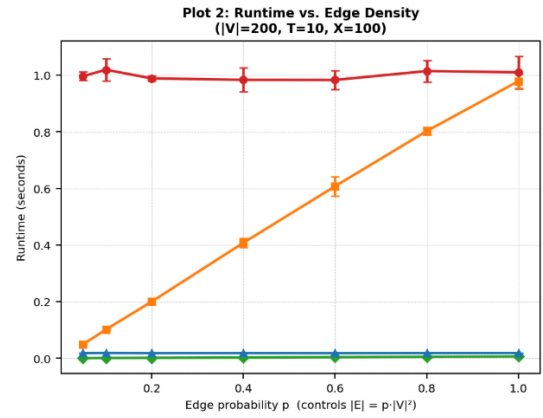


Fig. 2: Runtime vs. p (edge density)

Fig. 2: Runtime vs. p (edge density)

Fig. 3 (Runtime vs. X): As predicted, Both MC curves grow linearly in X . MC+Matrix is always $|V|^2/|E| \approx 10\times$ slower than MC+List at $p=0.1$. The two DP lines are flat for all X values, confirming DP runtime is completely independent of the number of simulations.

Fig. 4 (Runtime vs. T, log-log): The MC curves have an empirical slope ≈ 2 and DP curves a slope ≈ 1 , in exact agreement with $O(T^2)$ vs $O(T)$.

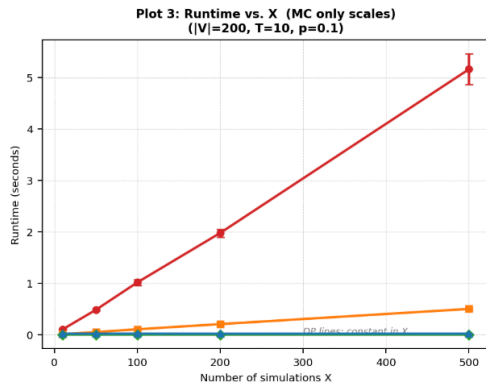


Fig. 3: Runtime vs. X (simulations)

Fig. 3: Runtime vs. X (simulations)

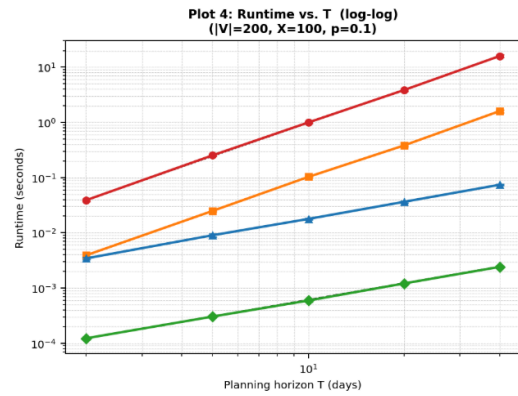


Fig. 4: Runtime vs. T (log-log)

Fig. 4: Runtime vs. T (log-log)

4. Reflection

The results agree with all four theoretical predictions I made earlier. The clearest confirmation is Fig. 4, where the log-log slopes match $O(T^2)$ vs $O(T)$ exactly, MC pays the T^2 cost from re-simulating from scratch for each outer day t , while DP reuses the previous column. Fig. 2 confirms the representation difference, matrix variants stay flat as p increases since they always scan the entire $|V|$ row regardless of how many edges exist, while list variants grow as p increases because more edges means longer adjacency lists to traverse. At $p = 1.0$, the MC list-matrix gap closes, as the adjacency list now traverses nearly all $|V|$ neighbours anyway. Fig. 3 confirms the X factor exists only in MC, DP lines are flat because it calculates the exact answer in a single pass with no simulations.

Task D requires to calculate risk scores for all residents and rank them in order to fairly distribute a limited number of vaccine doses over the full planning horizon. This requires accuracy and consistency, two properties MC fundamentally lacks. MC produces different risk estimates on every run due to simulation variance. This means the same resident could be ranked higher in one run and be excluded in another, which creates arbitrary unfairness in the allocation. Whereas DP, being deterministic, produces identical scores every time, ensuring the ranking is stable and defensible.

Speed also matters at scale. A real city network is large and sparse, many residents, but they're connected to only a small fraction of others. DP + adjacency list directly exploit this, its $O(T \cdot |E|)$ cost scales with actual connections, not the full $|V|^2$ population grid. Fig. 1 has demonstrated the fact that DP + List runs about 1000 $\times$ faster than MC + matrix at $|V|=700$, and as the city grows, this gap widens further. Ultimately, DP only needs to be run once to output an exact answer. In contrast, MC needs to be run X times per query to get rid of the noise and it still has variance. DP + adjacency list's exactness is not merely a performance benefit, it is an ethical one. Thus DP + adjacency list has proven to be the best choice.

If transmission rates or connections update daily, the DP table becomes stale and must be recomputed from scratch each day. For short T , we still prefer DP + adjacency list because recompute of $O(T \cdot |E|)$ is manageable. When updates are very frequent and approximate estimates are acceptable, MC with small X can be considered, but DP + list is still the better choice whenever accuracy matters.

Task D: Antiviral Allocation (10 marks, 2 pages)

Algorithm Design

To solve this problem we use a top-down memoized approach to the 0/1 Knapsack problem. Each resident V_i has a dosage cost $c_i \in \mathbb{Z}^+$ and a benefit score $b_i = r_i$ directly inherited from their infection risk in Task B. The general problem is to select which residents to vaccinate to maximize total benefit subject to the dose budget C . If benefits are equal, the selection of fewer doses wins.

In order to solve this matter, I used a 2D memo table $\text{memo}[i][c]$ of size $(n+1) \times (C+1)$, where $i \in [0, n]$ is the number of residents considered, and $c \in [0, C]$ is the remaining dose capacity. Each cell contains a tuple (benefit, doses_used), and the best result is obtained using first i residents with doses available. All cells are initially None.

The base case is $\text{memo}[0][c] = (0.0, 0)$ for all $c \in [0, C]$. However, If there are no residents to be chosen from, then both benefit and doses equals zero.

From there, I call a recursive function, $\text{solve}(i, c)$, which returns the best selection using only the first i residents with c doses remaining. Starting from $\text{solve}(n, C)$, This function does not calculate the whole table, only the two subproblems that it needs to solve, namely $\text{solve}(i-1, c)$ if resident i is skipped, and $\text{solve}(i-1, c-c_i)$ if they are included. These subproblems recursively retrieve only what is needed, leaving untouched cells as None. If a state has already been computed, the function returns the cached value, so that no unnecessary computation is done, maximising efficiency.

For each unvisited cell, two options are looked at:

- Skip V_i : $\text{memo}[i-1][c]$ remains the same.
- If $c_i \leq c$, include V_i : take $\text{memo}[i-1][c-c_i]$ and add b_i and c_i to it.

Then whichever gives higher benefit is stored. If they tie, the one with less doses used is retained. benefit scores are floats so no rounding is needed for comparisons.

Once $\text{solve}(n, C)$ is complete, $\text{memo}[n][C]$ will be assigned with the final answer. Then the selected residents are obtained by going backwards from row n to row 1 with a pointer c starting at C . At each row i , If $\text{memo}[i][c] \neq \text{memo}[i-1][c]$, then we add resident V_i to the result and decrease c by c_i . There is no extra computation needed since backtracking only reads cells that were filled in the recursive phase. Finally, the list is reversed to get back the original order.

Complexity Analysis

In the worst case, my top-down approach visits all $(n+1)(C+1)$ cells, giving $O(n \times C)$ time. Where each cell takes $O(1)$ to fill, so just two lookups, one addition, and a couple of comparisons. And backtracking at most n rows at $O(1)$ each, which adds $O(n)$ extra, but small enough to ignore.

Time complexity: $O(n \times C)$, Space complexity: $O(n \times C)$

But in practice, top-down is more efficient than bottom-up for this task. Only $\text{solve}(n, C)$ is called at the top so the recursion is limited to cells it actually needs, while cells that are never touched stay as None, so that we can skip them entirely. This leaves a significant portion of cells untouched for most inputs. In contrast, the Bottom up approach would fill up every single cell whether it is needed or not. That is wasteful and exactly why I chose top down instead. Top-down solves the minimal set of subproblems needed to get the correct answer which is much more efficient, and it also directly satisfies the requirement of computing minimal subproblems.

The n factor comes from looping over all n residents. C comes from checking every capacity value from 0 to C for each one. This is pseudo-polynomial, it grows with the actual size of C , not the number of bits needed to encode it. In practice this is fine as dosage costs are integers and C is bounded. Space can be reduced to $O(C)$ by retaining only two rows, but backtracking will not be possible.

Extension 1: Triage

Modified Algorithm

Suppose residents are divided into K tiers, $T_1, T_2, \dots, T_K$, ordered from most to least vulnerable. We give doses to the most vulnerable tier first, and then pass any leftover doses to the next tier. This is achieved by executing the standard knapsack DP, K times in sequence, with a shared dose counter initialised at C . Each tier's DP uses the doses allocated to it, and the remaining dose count is passed down to the next tier.

The tiers are disjoint, and $\sum_k |T_k| = n$, so the total work over all K runs is still $O(n \times C)$ as in the single-tier case. After backtracking, the memo table for each tier can be deleted.

Counterexample — Why Ignoring Tiers Fails

Let $C = 5$. V_1 (Tier 1, benefit 0.40, cost 4), V_2 (Tier 2, benefit 0.50, cost 2), V_3 (Tier 2, benefit 0.50, cost 3).

With no tiers, the flat DP picks $\{V_2, V_3\}$ for a total benefit of 1.00, skipping V_1 entirely, because of how Tier 2's cost is 2 cheaper than Tier 1, and their residents rank higher together. But V_1 is the most vulnerable resident and must be protected first.

With tiers, Tier 1 runs first and picks $\{V_1\}$ (benefit 0.40, cost 4) and 1 dose is left. No Tier 2 resident can be vaccinated as neither V_2 nor V_3 fits within the remaining 1 dose. The total benefit is smaller, but the most vulnerable resident is guaranteed treatment. And this is the point of triage, protecting the most at-risk people is a higher priority that comes before maximising total benefit.

Extension 2: Interdependent Vaccinations

Why Independence Is Necessary

My DP works by re-using previously calculated results. When filling $\text{memo}[i][c]$ it looks at $\text{memo}[i-1][c-c_i]$ and adds a fixed benefit b_i . This only works if b_i is the same regardless of who else has been chosen. If the vaccination of V_i reduces the risk of its neighbors, then the benefit of vaccination to the neighbors depends on whether V_i was picked. Stored results are no longer reusable, optimal substructure fails, and my DP gives wrong answers.

Counterexample

Let $C = 5$. V_1 connects to V_2 and V_3 . If V_1 is vaccinated, the value of each neighbour is reduced by 0.25. V_1 costs 3 doses, with benefits 0.80. V_2 and V_3 cost 2 doses each, base benefit 0.55 (0.30 if V_1 vaccinated).

The independent DP assumes all benefits are fixed and selects $\{V_2, V_3\}$ which gives a total benefit of 1.10. But V_2 and V_3 contribute only 0.30 each in this case, so the true combined benefit of V_2 and V_3 is only 0.60. When the truth is it's better to vaccinate only V_1 at 0.80. The DP gets the wrong answer because it never considers how choosing V_1 changes everything else.

Why the True Problem Is Much Harder

If the benefits depend on who else is picked, then you can't break the problem into smaller reusable subproblems anymore. The value of adding any resident depends on the whole set chosen up until now, so results stored previously cannot be relied upon. No DP table will fix this.

The only way to be sure you are getting the real best answer is to search all 2^n possible subsets, which is an $O(n \times 2^n)$ process. This makes the problem NP-hard. It is hard to even get a good approximate solution, unless the benefit function has a nice property called monotone submodularity, which means that marginal benefits are always positive and decrease as we add more residents. In this case, a greedy approach can achieve $(1-1/e) \approx 0.63$ of the true optimum, but still costs $O(n^3)$, far more than the $O(n \times C)$ independent DP. In general, without that property, even a rough approximation isn't guaranteed.

